

Vrije Universiteit Amsterdam



Universiteit van Amsterdam



Literature Study

A Literature Study on Large Language Models for Software Testing and Fault Localization

Author: Ziniu Jin

1st supervisor: Ana Opreescu
daily supervisor: Supervisor Name
2nd reader: Second Reader Name

*A literature study submitted as part of the requirements for
the joint VU-UvA Master of Science degree in Computer Science*

May 16, 2026

Abstract

This literature review summarizes recent research on software testing challenges and LLM-based testing techniques. It analyzes 25 papers covering topics such as flaky tests, UI testing vulnerabilities, fault location, vulnerability detection, test remediation, and testing issues in CI/CD, IoT, microservices, and industrial environments. Overall, modern software testing problems are often not single-technology issues but are closely related to specific programming languages, testing frameworks, system architectures, and runtime environments.

This review also finds a shift in research methodology. Early research relied more on test execution, coverage data, repetitive runs, and traditional fault location techniques; while recent research has begun to utilize LLM more for prediction, classification, remediation, and agentic testing that more closely resembles real-world development processes. Although these methods have shown potential, most LLM-based methods are still primarily evaluated on academic benchmarks or relatively isolated tasks, lacking sufficient validation in real-world development environments. Therefore, this paper argues that future research should focus on LLM-based fault localization in real-world repository settings, especially by combining bug reports, source-code context, and verified bug-fixing commits for evaluation.

Contents

1	Introduction	3
2	Methodology	3
2.1	Research Questions	3
2.2	Literature Search Strategy	4
2.3	Study Selection Criteria	5
2.3.1	Inclusion Criteria	5
2.3.2	Exclusion Criteria	5
2.4	Snowballing	6
2.5	Study Selection Results	6
3	Results	17
3.1	RQ1: Testing Challenges and Root Causes	17
3.2	RQ2: Detection and Analysis Approaches	18
3.3	RQ3: Use of Large Language Models	20
3.4	RQ4: Evaluation and Benchmarking	21
3.5	RQ5: System Context and Practical Implications	23
4	Related Work	24
5	Discussion	24
5.1	Overview of the Current Research Landscape	24
5.2	Research Gaps Across Methodological Themes	25
5.3	Glasswing as an Emerging Practical Reference	25
5.4	Consolidated Research Gaps	26
6	Responsible Use of AI Tools	27
7	Conclusion	28

1 Introduction

As software systems continue to expand in scale and architectural complexity, software testing plays an increasingly critical role in ensuring system quality and reliability. Within modern software development processes, continuous integration and continuous delivery (CI/CD) have become mainstream practices. However, in such environments, testing challenges exhibit greater complexity and diversity. For instance, integrating UI testing into CI/CD pipelines significantly impacts workflow stability, test maintenance costs, and development efficiency [1]. Furthermore, empirical studies on open-source IoT platforms reveal that issues such as insufficient test coverage, difficulties in environment simulation, and high maintenance costs remain prevalent in real-world system environments [2].

Among numerous testing challenges, test instability—commonly referred to as *flaky tests*—has emerged as a major research focus. Flaky tests are test cases that exhibit non-deterministic result fluctuations without any modification to the underlying program. Recent studies have systematically analysed sequence-dependent flaky tests in JavaScript systems, revealing the significant influence of shared state and test execution order on stability [3].

Concurrently, as system complexity increases, defect localisation and vulnerability detection problems have become increasingly prominent. Traditional spectral fault localisation techniques are being enhanced through the integration of compiler information and structural features to improve localisation accuracy [4]. In recent years, large language models (LLMs) have rapidly gained traction in software engineering research, demonstrating potential in defect localisation [5] and security vulnerability detection [6]. Moreover, LLMs have been applied to Web UI test repair tasks, where attribute prioritisation strategies and prompt engineering techniques significantly enhance repair effectiveness [7].

Within distributed and microservices architectures, system integration-related faults have become a significant research focus. Existing studies have systematically categorised integration-related faults in microservices testing, providing a structured perspective for understanding testing challenges in complex systems [8].

Despite the growing body of research on flaky tests, UI test maintenance, defect localisation, and vulnerability detection, existing work remains fragmented across multiple subdomains and lacks comprehensive synthesis and structured comparison. To address this gap, this paper conducts a systematic literature review. By systematically retrieving, screening, and categorising relevant studies, this work aims to provide a coherent research landscape and a structured reference framework to guide future research in intelligent software testing.

2 Methodology

This study follows a systematic literature review process to collect, screen, and synthesize research on software testing challenges and LLM-based testing techniques. Following the guidelines proposed by Kitchenham et al. [9], the overall process includes literature retrieval, literature screening, and the organisation and classification of research content, aiming to form a comprehensive understanding of the current research status.

In the specific implementation, a candidate literature set is first obtained through a pre-defined search strategy. Then, the final research literature included in the analysis is determined based on clear screening criteria. The selected research is categorised and summarised from the perspectives of research topics and issues of concern, providing a foundation for subsequent analysis and discussion.

2.1 Research Questions

Based on the scope of this systematic literature review and the selected primary studies, we define the following research questions to guide the analysis.

- **RQ1 (Testing Challenges and Root Causes):** What types of software testing challenges are reported in the literature, and what root causes are identified?

This question aims to identify and categorize testing challenges such as flaky tests, UI test fragility, state-dependent failures, performance instability, and security vulnerabilities.

- **RQ2 (Proposed Detection and Analysis Approaches):** What approaches are proposed to detect, localize, or analyze software testing problems?

This includes spectrum-based fault localization, mutation-based techniques, fuzzing approaches, compiler-assisted filtering, and LLM-based detection or reasoning methods.

- **RQ3 (Use of Large Language Models):** How are large language models (LLMs) applied to testing-related tasks, and what design strategies are adopted (e.g., fine-tuning, few-shot learning, prompt engineering)?

This question investigates architectural choices, adaptation strategies, and evaluation results of LLM-based approaches in tasks such as fault localization, vulnerability detection, UI test repair, and flaky test classification.

- **RQ4 (Evaluation and Benchmarking):** How are proposed approaches evaluated, and what datasets, metrics, and experimental designs are used?

This includes benchmark selection (e.g., Defects4J, CVEFixes, FlakyCat, IDoFT), performance metrics (e.g., Top-k accuracy, F1 score), and comparison against state-of-the-art techniques.

- **RQ5 (System Context and Practical Implications):** In which system contexts (e.g., microservices, CI/CD pipelines, IoT platforms, distributed systems) are testing challenges studied, and what practical implications are reported?

This question explores the applicability of testing techniques in real-world settings and examines industrial challenges and integration considerations.

These research questions structure the search strategy, study selection, data extraction, and thematic synthesis presented in the following sections.

2.2 Literature Search Strategy

This study employs a systematic literature search strategy to ensure comprehensive coverage and reproducibility of relevant research. The search primarily uses mainstream academic databases in software engineering and computer science, including *IEEE Xplore* and the *ACM Digital Library*, supplemented by *Google Scholar*.

The search topics revolve around software testing, defect analysis, test automation, and intelligent methods, with a particular focus on recent progress in combining Large Language Models (LLMs) with software testing techniques. To improve the relevance of search results, multiple keyword combinations were used, and keywords were iteratively refined based on initial search outcomes to optimize both coverage and precision.

Keywords. Keywords used in the search include, but are not limited to:

- software testing
- flaky test
- fault localization
- vulnerability detection

- UI testing
- large language model
- automation

Boolean combinations. Boolean operators (e.g., AND/OR) were applied to cover relevant research expressed with different terminology. Example combinations include:

- ("software testing" OR "test automation") AND "large language model"
- "flaky test" AND ("large language model" OR LLM)
- "fault localization" AND ("large language model" OR LLM)
- ("vulnerability detection" OR "security testing") AND ("large language model" OR LLM)
- "UI testing" AND ("large language model" OR LLM)

Scope restrictions. To ensure quality and consistency, the search was limited to peer-reviewed academic papers (journal articles, conference papers, and workshop papers) published in English. All retrieved results were systematically recorded and incorporated into the subsequent screening and analysis steps.

2.3 Study Selection Criteria

To ensure scientific rigor and reliability, this study established explicit inclusion and exclusion criteria during the literature screening phase. All retrieved studies were systematically evaluated according to these criteria.

2.3.1 Inclusion Criteria

Studies were included if they met the following conditions:

- Published as peer-reviewed journal articles, conference papers, or workshop papers;
- Related to software testing, defect analysis, fault localization, vulnerability detection, UI testing, or test automation technologies;
- Proposed methods for detecting, analyzing, locating, or fixing testing-related problems;
- For studies involving Large Language Models (LLMs), clearly described the model application approach (e.g., fine-tuning, few-shot learning, or prompt engineering) as well as the experimental evaluation process;
- Provided a clear problem definition, methodological description, and experimental design to support classification, comparison, and synthesis.

2.3.2 Exclusion Criteria

Studies were excluded if they met any of the following conditions:

- Non-peer-reviewed publications, such as preprints, technical blogs, industry reports, or white papers;
- Low relevance to the research topic or only indirectly related to software testing;

- Purely conceptual discussions without empirical validation or methodological details;
- Duplicate publications of the same study (in which case, only the most complete or most recent version was retained).

The studies selected according to the above criteria constitute the core research foundation for the subsequent analysis and synthesis presented in this work.

2.4 Snowballing

Since the initial search was confined to a single conference venue (IEEE ICST), the coverage of the resulting literature pool was inherently limited. To improve the comprehensiveness of the search and to satisfy the quality criteria for search coverage proposed by Kitchenham et al. [9], we supplemented the initial search with a round of forward and backward snowballing.

Backward Snowballing

The backward snowball sampling method was applied to the reference lists of nine seed studies. Each seed study was manually reviewed to identify all cited literature, and all candidate studies were then assessed against the inclusion and exclusion criteria defined in Section 2.3. This process identified a total of 18 candidate studies, of which eight were ultimately included following screening.

Forward Snowballing

A forward snowballing approach was used to identify follow-up studies citing the aforementioned nine seed publications via Google Scholar, with a search cut-off date of March 2026. All identified candidate publications were screened against the criteria set out in Section 2.3. This process identified a total of seven candidate publications, of which three were ultimately included following screening.

Table 1: Summary of literature selection across all search phases.

Search Phase	Candidates Identified	Studies Included
Initial search	–	9
Backward snowballing	18	8
Forward snowballing	7	3
Extended venue search	5	5
Total	30	25

2.5 Study Selection Results

Table 2: Included studies from the initial search.

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[1]	UI Testing; CI/CD Integration	Qualitative study; developer survey and interviews	Identifies challenges (fragility, maintenance cost), strategies (selective automation), and impacts of UI testing in CI/CD	Empirical insight into UI testing challenges and practices in CI/CD environments	RQ1, RQ5
[2]	IoT Platform Testing; Developer Perspectives	Mixed-method: repository mining (37K test methods) + survey of 80 developers	Significant lack of test coverage (test ratio < 0.5 for 65% of add-ons); unique IoT challenges around device compatibility	Reveals practical testing challenges and coverage limitations in IoT platforms	RQ1, RQ5
[3]	Flaky Tests; Test Order Dependency; JavaScript	JS-TOD reordering tool; 81 GitHub projects; manual root-cause analysis	55 order-dependent tests found; shared mocking state (42 tests) identified as previously unreported JS-specific root cause	Identifies root causes of order-dependent flaky tests in JavaScript systems	RQ1, RQ2
[10]	Flaky Test Detection; LLM Fine-Tuning; Few-Shot Learning	Comparative evaluation of fine-tuning vs. FSL (FlakyXbert); FlakyCat and IDoFT datasets	Fine-tuning achieves higher accuracy with sufficient data; FSL competitive at lower cost in resource-constrained settings	Compares LLM adaptation strategies for flaky test detection	RQ3, RQ4
[4]	SBFL; Compiler Information; Haskell	25 filtering rules (type info, AST, test results) applied to SBFL spectra; 11 Haskell programs	Combined filters improve TOP50 by up to 67.4% (DStar2); proximity to failure and non-unique types most effective	Enhances SBFL through compiler-assisted filtering strategies	RQ2, RQ4

Continued on next page

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[6]	Security Vulnerability Detection; LLM Evaluation	16 LLMs on 5,000 samples from 5 datasets; three prompting strategies (Basic, CWE, Dataflow)	Mean accuracy 62.8%, F1 0.71; dataflow prompt improves F1 by up to 0.18; smaller models can outperform GPT-4	Evaluates LLM effectiveness and prompting strategies for vulnerability detection	RQ2, RQ3, RQ4
[7]	Web UI Test Repair; LLM; Attribute Prioritization	ChatGPT for element matching; explanation validator for hallucination mitigation	EDIT DIS+ChatGPT fixes 122/139 statements (+186%); explanation validator reduces hallucination	Demonstrates LLM-based UI test repair and hallucination mitigation	RQ2, RQ3, RQ4
[5]	Fault Localization; LLM Fine-Tuning; Code Models	SG approach fine-tuning 13 LLMCs (7 architectures); Defects4J benchmark	UniXcoder achieves Top-1/3/5 of 50.6%/64.2%/72.3%, outperforming SOTA by 1.35×	Shows the effectiveness of fine-tuned code LLMs for fault localization	RQ2, RQ3, RQ4
[8]	Microservice Testing; Integration Faults; Taxonomy	SLR + 10 expert interviews; qualitative coding	Empirically validated taxonomy of integration-relevant faults in microservice systems	Provides a taxonomy of integration-related faults in microservice testing	RQ1, RQ5

Table 3: Included studies from backward snowballing.

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[11]	Flaky Tests; Root Cause Analysis	Empirical study; manual analysis of 201 flaky tests from open-source projects	10 root cause categories (async wait, concurrency, test order dependency, etc.); foundational taxonomy	Establishes a foundational taxonomy of flaky test root causes	RQ1, RQ2
[12]	Flaky Tests; Survey	Systematic mapping study; 172 papers analysed	Comprehensive taxonomy of flaky test causes, detection methods, and repair strategies	Synthesises prior research on flaky tests into a broad taxonomy	RQ1, RQ2
[13]	Flaky Test Classification; FSL; LLM	Siamese network + CodeBERT; evaluated on FlakyCat	Competitive classification accuracy with fewer labelled examples; validates few-shot LLM viability	Validates few-shot LLM-based classification for flaky tests	RQ3, RQ4
[14]	Flaky Test Prediction; LLM; Black-box	Black-box LLM predictor using code token sequences; no AST or execution traces	Predicts flaky tests without program analysis; high accuracy across diverse projects	Demonstrates black-box LLM prediction of flaky tests	RQ3, RQ4
[15]	SBFL; Tarantula	Colour-coded statement suspiciousness based on pass/fail coverage ratios	Introduces Tarantula, foundational SBFL formula	Provides a foundational SBFL technique used in later fault localization research	RQ2

Continued on next page

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[16]	SBFL; Ochiai Formula	Comparative evaluation of SBFL coefficients on Siemens benchmark	Ochiai consistently outperforms other SBFL formulas; established as standard baseline	Establishes Ochiai as a strong baseline in SBFL research	RQ2
[17]	Pre-trained Code LLM; Code Representation	Bimodal pre-training on NL-PL pairs; masked LM + replaced token detection	SOTA on code understanding tasks; widely adopted as backbone in FL, test repair, and vulnerability detection	Provides a widely used code LLM backbone for later testing-related tasks	RQ3
[18]	UI Test Fragility; Record/Replay; Root Cause	Empirical analysis of 2,923 broken test scripts across 8 web applications	Root causes of UI test breakage (DOM changes, timing, event handling); empirical basis for UI repair research	Provides empirical evidence on UI test fragility and breakage causes	RQ1, RQ2

Table 4: Included studies from forward snowballing.

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[19]	Flaky Test Repair; LLM; Fix Classification	LLM fine-tuned on flaky test-fix pairs; predicts fix category and generates code repair	Correctly predicts fix categories for 73% of flaky tests; generates compilable repairs	Extends LLM applications from detection to repair of flaky tests	RQ2, RQ3
[20]	Fault Localization; LLM; Test-Free	LLM-based FL using bug reports and code context; no test execution required	Competitive FL accuracy without test suite; opens new direction beyond SBFL	Introduces a test-free LLM-based direction for fault localization	RQ2, RQ3
[21]	LLM Testing Agents; Autonomous Testing	Framework for LLM-driven autonomous test agents; iterative generation, execution, and repair	Demonstrates feasibility of autonomous LLM testing agents; identifies key challenges	Broadens LLM use toward autonomous testing agents and workflows	RQ2, RQ3

Table 5: Included studies from extended venue search.

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[22]	Vulnerability Detection; Code Language Models; Benchmark Quality	Empirical evaluation of code language models on vulnerability detection; construction of PrimeVul dataset with improved labelling, de-duplication, and chronological splitting	Existing vulnerability datasets may overestimate model performance due to duplication and label noise; model performance drops sharply under the more realistic PrimeVul setting	Provides a cleaner benchmark for evaluating code language models in vulnerability detection	RQ2, RQ3, RQ4
[23]	Vulnerability Detection; Inter-Procedural Context; LLM-Assisted Analysis	VulnSC framework; retrieves called functions and uses LLM-generated semantic summaries to enrich vulnerability detection inputs	Adding inter-procedural semantic information improves vulnerability detection compared with function-level analysis alone	Shows that vulnerability detection benefits from broader program context beyond the target function	RQ2, RQ3, RQ4
[24]	LLM Testing Agents; Test Execution; Repository-Level Automation	LLM-based agent that installs dependencies, builds projects, runs tests, observes failures, and revises commands	Successfully executes test suites across diverse open-source projects and programming languages, showing that test execution itself can be automated with LLM agents	Introduces an agentic approach for practical test execution in arbitrary software projects	RQ2, RQ3, RQ5

Continued on next page

Citation	Research Topic	Method / Approach	Key Findings	Contribution	Relevance to Research Questions
[25]	LLM-Generated Tests; Flaky Tests; Database Systems	Empirical study of tests generated by GPT-4o and Mistral-Large-Instruct for industrial and open-source DBMSs	LLM-generated tests can be flaky; common causes include reliance on unspecified ordering, especially unordered collections	Shows that LLMs can introduce new test reliability problems when generating tests	RQ1, RQ3, RQ5
[26]	Industrial Flaky Tests; CI Environment; SAP HANA	Empirical analysis of SAP HANA test data over 12 weeks, including production test results and a dedicated flakiness experiment	Average test execution time shows the strongest correlation with flakiness rate; infrastructure load does not show a clear correlation	Provides industrial evidence on factors associated with flaky tests in large-scale database testing	RQ1, RQ4, RQ5

Table 6: Excluded studies from backward snowballing.

Citation	Research Topic	Method / Approach	Reason for Exclusion	EC	Notes
[27]	Flaky Tests; Test Order Dependency	Tool-based detection of order-dependent flaky tests	Content already covered by the foundational flaky test studies and survey	EC4	Redundant with B01 and B02 for the purpose of this review
[28]	Test Dependency; State Pollution	Detection of state-polluting tests in Java projects	Same general order-dependency topic as included flaky test studies; does not use LLM-based methods	EC4	Useful background, but not selected as a primary study
[29]	Flaky Tests; Root Cause Analysis	Empirical analysis of flaky test causes in software projects	Root-cause categories overlap with Luo et al.; industrial context is outside the main scope of the selected corpus	EC2	Excluded to avoid duplication of the flaky test taxonomy discussion
[30]	Flaky Test Prediction; Machine Learning	Traditional machine-learning-based flaky test prediction	Uses traditional ML rather than LLM-based analysis; related direction already represented by Flakify	EC2	Covered indirectly through B04
[31]	Spectrum-Based Fault Localization; Survey	Survey of SBFL techniques and formulas	Background survey only; no new detection or localization method selected for analysis	EC2	Core SBFL background already covered by Tarantula and Ochiai studies
[32]	Pre-trained Code Models; Code Representation	Pre-training approach for source code representation	Same general background category as CodeBERT; redundant for the LLM backbone discussion	EC4	CodeBERT was kept as the representative pre-trained code model

Continued on next page

Citation	Research Topic	Method / Approach	Reason for Exclusion	EC	Notes
[33]	Defect Prediction; Dataset	Dataset construction for software defect prediction	Dataset paper without a directly proposed testing, detection, or localization method	EC3	Excluded because it does not directly address the research questions
[34]	Vulnerability Detection; Graph Neural Networks	Deep-learning approach using graph-based code representations	Non-LLM deep-learning method; outside the LLM-focused scope of the review	EC2	Could be used as broader background but not as a primary study
[35]	Web UI Test Repair; DOM Similarity	DOM-similarity-based repair of broken web tests	Pre-LLM UI repair tool with limited relevance to the main analysis	EC2	Later UI repair and LLM-based repair studies provide closer coverage
[36]	IoT Bugs; Software Testing	Empirical study of bugs in IoT systems	IoT topic already covered by S2; study does not focus on LLM-based testing or repair	EC2	Excluded to keep IoT discussion focused on testing practices

Table 7: Excluded studies from forward snowballing.

Citation	Research Topic	Method / Approach	Reason for Exclusion	EC	Notes
[37]	Flaky Test Repair; LLM	LLM-based repair approach for flaky tests	Preprint and not peer-reviewed; overlaps with included LLM flaky test repair and testing-agent studies	EC1	Excluded due to publication status
[38]	Flaky Test Repair; Static Analysis	Static-analysis-based repair of flaky tests	Non-LLM repair approach; outside the main LLM-focused analysis	EC2	Related as background, but not selected as a primary study
[39]	Flaky Tests; Test Reordering	Heuristic-based test reordering for order-dependent flaky test detection	Non-LLM reordering method; topic already covered by included flaky test studies	EC2	Excluded to avoid overlap with S3, B01, and B02

3 Results

3.1 RQ1: Testing Challenges and Root Causes

What types of software testing challenges are reported in the literature, and what root causes are identified?

The primary studies report testing challenges across four distinct categories: flaky test instability, UI test fragility, integration faults in distributed and microservice systems, and platform-specific testing difficulties in IoT environments.

Flaky Test Instability. Flaky tests—tests that may pass or fail without any change to the production code—are the most frequently discussed challenge in our corpus. Seven primary studies focus on this issue directly (B01, B02, S3, S4, B03, B04, and F01).

Luo et al. [11] (B01) presented one of the earliest and most influential empirical studies on flaky tests. By manually analysing 201 flaky test fixing commits from 51 open-source projects, they proposed ten root-cause categories. The most common were *Async Wait*, *Concurrency*, and *Test Order Dependency*. One important result is that 78% of flaky tests were already flaky when they were first introduced. This suggests that tools aimed at early detection could prevent many later failures. They also found that 74% of order-dependent flaky tests were fixed by cleaning shared state between test runs, showing that some causes of flakiness can be addressed in a straightforward way.

The seed papers also highlight ecosystem-specific causes of flakiness. Hashemi et al. [3] (S3) studied 81 open-source Jest projects using the JS-TOD tool, which systematically reordered tests and reran them multiple times. They found 55 order-dependent tests across 10 projects. Among these, 42 cases (76%) were caused by *shared mocking state*, a cause not reported in earlier Java or Python studies. This suggests that flaky test causes can depend on the language and framework being used.

UI Test Fragility. UI test fragility is the second major challenge category and is addressed by three primary studies (S1, B08, S7).

Hammoudi et al. [18] (B08) carried out one of the first detailed studies of why record/replay tests break in web applications. They analysed 453 application versions and identified 1,065 separate breakages across eight web systems. Based on these results, they proposed five main categories of breakage causes: *Locators*, *Values*, *Page Reloading*, *User Session Times*, and *Popup Boxes*. The most common problem was changes to DOM element attributes, which frequently caused recorded locators to fail. Their findings help explain why UI tests often require continuous maintenance.

Gan et al. [1] (S1) examined UI testing from a developer and workflow perspective by studying GitHub projects that used Selenium, Playwright, or Cypress in GitHub Actions. They found that many developers view UI testing as costly to maintain and difficult to keep stable in CI/CD pipelines. Common problems included environment setup, failures caused by DOM changes, and difficulty reproducing issues across different execution environments. As a result, some open-source projects avoid extensive UI test automation altogether.

Integration Faults in Microservice Systems. Gregor et al. [8] (S9) studied testing problems in microservice systems, where challenges go beyond individual test cases. Using a literature review together with ten interviews with industry experts, they developed a taxonomy of integration-related faults in microservice architectures.

Their results show that failures often come from technical issues such as API mismatches, incompatible message formats, and network latency. However, they also identified organisational causes, including weak coordination between teams and unclear service ownership. This means

that testing problems in microservice systems are partly sociotechnical and cannot always be solved through traditional testing tools alone.

IoT Platform Testing Challenges. Rodriguez-Cardenas et al. [2] (S2) examined testing practices in open-source IoT platforms through repository mining and a survey of 80 developers. Their dataset included around 37,000 test methods from OpenHAB and HomeAssistant.

They found clear signs of low test coverage. For example, 65% of HomeAssistant’s 937 additions had a test ratio below 0.5, while OpenHAB’s overall test ratio was only 0.04. This indicates a large gap between platform size and available tests.

The study also identified several IoT-specific difficulties. These include cross-device and cross-platform compatibility testing, debugging firmware updates that require access to physical devices, and testing behaviour in low-power modes, where assumptions such as stable connectivity or predictable timing no longer hold. These constraints help explain why testing remains difficult in this domain.

A common pattern across all categories is that testing problems are highly context-dependent. Causes that appear in one ecosystem may not appear in another, and solutions that work for unit tests may not transfer to UI, microservice, or IoT settings. This helps explain why the literature contains a wide range of detection and analysis approaches, discussed next in RQ2.

3.2 RQ2: Detection and Analysis Approaches

What approaches are proposed to detect, localize, or analyze software testing problems?

The primary studies describe four main groups of approaches: spectrum-based fault localization (SBFL), flaky test detection and repair, LLM-based fault localization, and performance fuzzing. Together, these methods address the testing challenges identified in RQ1 from different technical perspectives.

Recent studies show that flakiness can also appear in LLM-generated tests. Berndt et al. [25] (E04) study tests generated by GPT-4o and Mistral-Large-Instruct for industrial and open-source database systems. They find that these tests can be flaky, often because they rely on unspecified behaviour such as the order of unordered collections. This suggests that LLMs may not only help with testing problems, but can also introduce new ones.

Berndt et al. [26] (E05) study flaky tests in SAP HANA. They find that average test execution time has the strongest correlation with flakiness rate, while infrastructure load does not show a clear correlation. This suggests that, in large systems, flakiness is also shaped by test duration and execution complexity.

Spectrum-Based Fault Localization (SBFL). SBFL is the most established line of work in our corpus and is represented by three primary studies (B05, B06, S5).

Jones et al. [15] (B05) introduced Tarantula, one of the earliest and most influential SBFL techniques. Tarantula assigns each program statement a suspiciousness score based on how often it is executed by passing tests compared with failing tests. Statements mainly executed by failing tests receive higher scores and are shown in red, while statements more associated with passing tests are shown in green. Their evaluation on the Space program showed that the approach can guide developers toward faulty code and reduce the amount of code that must be inspected manually [15].

Abreu et al. [16] (B06) extended this line of work by comparing several SBFL similarity coefficients on the Siemens benchmark and the Space program. Their results showed that the Ochiai coefficient performed better than the other evaluated formulas, including Tarantula, across different testing conditions. They also reported that strong diagnostic results were possible even with relatively small test suites and imperfect observations. On average, more than 80% of the

code could be ruled out from inspection [16]. Because of these results, Ochiai later became a common baseline in SBFL research.

Applis et al. [4] (S5) focus on a common problem in SBFL for larger systems: execution spectra often contain many program elements unrelated to the real fault, which weakens ranking quality. To address this, they proposed 25 filtering rules based on compiler information, including type information, AST properties, and test outcomes. These rules remove less useful elements before suspiciousness ranking is applied. In experiments on 11 real-world Haskell programs, several filters noticeably improved ranking results. The best combinations increased TOP50 performance by 45.5% for Ochiai, 67.4% for DStar2, and 45.5% for Tarantula [4]. The most useful rules were related to proximity to failing expressions and the presence of non-unique types. The authors also noted that some faults caused by code generation or system-level behaviour could not be handled by SBFL, showing that spectrum-based methods have clear limits.

Flaky Test Detection and Repair. Four studies address flaky test detection and repair through complementary strategies, ranging from dynamic reordering to static LLM-based prediction and automated code repair (S3, B03, B04, F01).

Hashemi et al. [3] (S3) propose JS-TOD, a dynamic detection tool that systematically reorders tests at three granularity levels—test suites, individual tests, and describe blocks—applying 10 unique reorders and 10 reruns per level. Applied to 81 open-source Jest projects, JS-TOD detects 55 order-dependent flaky tests across 10 projects, with 42 caused by shared mocking state [3].

Akli et al. [13] (B03) target *categorisation* rather than binary detection. FlakyCat combines CodeBERT code representations with a Siamese network-based few-shot learning classifier to predict the root cause category of a flaky test from its source code alone, without requiring test execution. Evaluated on 343 flaky tests from open-source Java projects, FlakyCat achieves a weighted F1 score of 70% [13].

Fatima et al. [14] (B04) introduce Flakify, a black-box LLM-based predictor that determines whether a test is flaky by analysing its source code token sequence only, requiring neither access to production code, nor test re-execution, nor manually defined features [14].

Fatima et al. [19] (F01) extend the problem further from detection to *repair*. FlakyFix proposes a two-stage framework: a code model (CodeBERT or UniXcoder) first predicts which of 13 fix categories applies to the flaky test; this predicted category is then injected into the prompt of GPT 3.5 Turbo to guide automated test code repair. The fix category label raises the mean CodeBLEU score from 75% to 81%, and the authors estimate that roughly 51%–83% of generated repairs can be expected to pass [19].

LLM-Based Fault Localization. Two studies explore the use of large language models for fault localization, with the shared goal of reducing reliance on traditional test execution signals (S8, F02). Ji et al. [5] (S8) propose a sequence-generation approach that fine-tunes 13 code language models from seven architectures on the Defects4J benchmark. Among the evaluated models, UniXcoder achieved the best overall results, localizing faults within Top-1, Top-3, and Top-5 ranks for 50.6%, 64.2%, and 72.3% of 1,291 buggy methods respectively. This outperformed the strongest previous learning-based baseline by up to $1.35\times$ [5]. The study also found that encoder-based architectures generally performed better than decoder-based models, likely because bidirectional context is more suitable for localization tasks than left-to-right generation.

Yang et al. [20] (F02) go a step further by removing the need for test cases entirely. Their approach, LLMAO, adds lightweight bidirectional adapter layers to frozen pre-trained language models ranging from 350M to 16B parameters. These adapters are trained to predict line-level fault suspiciousness directly from source code. On Defects4J, LLMAO improved Top-1 performance over existing machine-learning baselines by 2.3% to 54.4%, and Top-5 performance by 14.4% to 35.6% [20]. The authors further observed that localization accuracy increased with

model size, suggesting that larger models may continue to improve this style of test-free fault localization.

LLM-Based Test Execution. Bouzenia and Pradel [24] (E03) study a practical problem often skipped in benchmark-based work: before test results can be analysed, the tests must first run. ExecutionAgent uses an LLM agent to install dependencies, build projects, run tests, observe failures, and revise commands. This shows that LLM-based testing is not only a prediction task, but also an environment and workflow problem.

3.3 RQ3: Use of Large Language Models

How are large language models (LLMs) applied to testing-related tasks, and what design strategies are adopted?

The primary studies apply LLMs to a range of testing tasks using three main strategies: fine-tuning, few-shot learning, and prompt engineering. This section discusses each strategy and the studies that adopt it.

Fine-tuning. Fine-tuning involves training a pre-trained model on a task-specific dataset to adapt it to a new task. Several studies in our corpus use this approach.

CodeBERT [17] (B07) is the most widely used backbone model in this area. It is pre-trained on both natural language and programming language data from six languages using masked language modeling and replaced token detection. It achieves state-of-the-art results on code search and code documentation generation, and is used as a backbone in several other studies in this corpus (S4, B03, F01).

Ji et al. [5] (S8) apply fine-tuning to fault localization. They fine-tune 13 LLMs across 7 architectures on the Defects4J benchmark using a sequence generation approach. UniXcoder achieves the best results, correctly identifying the fault in the Top-1, Top-3, and Top-5 predictions for 50.6%, 64.2%, and 72.3% of 1,291 methods, outperforming the best prior approach by up to 1.35 $\times$. Encoder-based architectures consistently outperform decoder-based ones for this task.

Fatima et al. [19] (F01) use fine-tuning as part of a two-stage repair framework. A fine-tuned code model (CodeBERT or UniXcoder) first predicts which of 13 fix categories applies to a flaky test. This predicted category is then added to the prompt of GPT 3.5 Turbo to guide code repair. Including the fix category raises the mean CodeBLEU score from 75% to 81%, and the authors estimate that roughly 51%–83% of the generated repairs can be expected to pass.

Few-shot learning. Few-shot learning allows a model to learn from a small number of examples, which is useful when labelled data is scarce.

Akli et al. [13] (B03) propose FlakyCat, which combines CodeBERT with a Siamese network to classify flaky tests by root cause category using only a small set of labelled examples. Evaluated on 343 flaky tests from open-source Java projects, it achieves a weighted F1 score of 70%. Async Wait, Unordered Collections, and Time-related categories are classified well, while Concurrency is harder to predict.

More and Bradbury [10] (S4) compare fine-tuning and few-shot learning on the FlakyCat and IDoFT datasets. Their results show that fine-tuning achieves higher accuracy when enough data is available, while few-shot learning provides competitive accuracy at lower cost, making it more suitable for teams with limited labelled data. Both approaches are viable depending on the resources available.

Prompt engineering. Prompt engineering adapts LLM behaviour through input design without changing model weights. It requires no training data and is the most flexible strategy.

Khare et al. [6] (S6) evaluate 16 LLMs on 5,000 code samples from five security datasets covering 25 vulnerability classes. They compare three prompting strategies: a basic prompt asking whether the code is vulnerable, a CWE-specific prompt targeting a particular vulnerability class, and a dataflow-based prompt that asks the model to reason about sources and sinks before making a prediction. The dataflow-based prompt improves F1 by up to 0.18 on real-world datasets. Overall, the models achieve a mean accuracy of 62.8% and F1 of 0.71. Interestingly, smaller models such as Qwen-2.5-14B outperform GPT-4 on real-world datasets, suggesting that model size does not always correlate with detection performance.

Xu et al. [7] (S7) apply ChatGPT to Web UI test repair. Prior repair tools tend to prioritise certain DOM attributes such as XPath during element matching, which can introduce bias. Their approach uses an existing tool for initial matching and then passes the candidate list to ChatGPT for a second matching step using all available attributes. To reduce hallucination, they design an explanation validator that checks whether ChatGPT’s explanation for its selection is consistent. If not, a self-correction prompt is sent. The best combination, EDIT DIS + ChatGPT, fixes 122 out of 139 broken statements, a 186% improvement over EDIT DIS alone.

Feldt et al. [21] (F03) take a broader view and propose a taxonomy of LLM-based testing agents organised by degree of autonomy. At the lower end, the LLM acts as an interactive assistant that responds to developer questions. At the higher end, the agent plans and executes testing tasks on its own. The authors note that LLM hallucination, usually seen as a problem, can sometimes be useful in testing because it generates unexpected inputs that human testers might not think of. Key limitations they identify include context length constraints, lack of memory across sessions, and the difficulty of integrating with existing testing tools.

Ding et al. [22] (E01) show that benchmark quality strongly affects LLM-based vulnerability detection results. They introduce PrimeVul to reduce label noise, duplicated samples, and unrealistic data splits. Their results suggest that earlier vulnerability detection benchmarks may have overestimated model performance.

Wu et al. [23] (E02) focus on the lack of program context in function-level vulnerability detection. VulnSC retrieves called functions and uses LLM-generated summaries to enrich the input. Their results suggest that broader inter-procedural context improves vulnerability detection.

3.4 RQ4: Evaluation and Benchmarking

How are proposed approaches evaluated, and what datasets, metrics, and experimental designs are used?

The primary studies use different datasets and metrics depending on the task. This section organises the evaluation practices by task area: fault localization, flaky test detection and repair, vulnerability detection, and UI test repair.

Fault Localization. Fault localization studies use two main benchmarks and one shared metric family.

Jones et al. [15] (B05) evaluate Tarantula on the Space program, a C interpreter with 9,564 lines of code and 33 faulty versions. The experiment runs 1,000 randomly generated test suites against 20 single-fault versions and 40 multi-fault versions. The metric is qualitative: the proportion of faulty statements that fall in the reddest colour partitions across all test suites. This measures how well the visualisation directs developer attention toward the correct location.

Abreu et al. [16] (B06) use the Siemens benchmark—a widely used collection of small C programs with known faults—along with the Space program. The key metric is *diagnostic accuracy*, defined as the proportion of code that can be exonerated after diagnosis. Ochiai

exonerates over 80% of all code on average, even with a limited number of test cases or low-quality error observations. The study also validates SFL in an industrial setting using embedded LCD television software from NXP Semiconductors.

Applis et al. [4] (S5) use TOP50 and TOP100 as their primary metrics, measuring whether the fault appears within the top 50 or top 100 ranked expressions. The dataset is 11 real-world Haskell programs. This setup differs from the Siemens benchmark and focuses more on ranking precision in larger programs.

Ji et al. [5] (S8) and Yang et al. [20] (F02) both use Defects4J as their primary benchmark, which is the most widely adopted Java fault dataset. S8 evaluates on 1,291 methods from Defects4J v1.2 and v2.0 using Top-1, Top-3, and Top-5 metrics—whether the correct fault location appears within the top 1, 3, or 5 predictions. F02 uses the same benchmark and metrics, comparing LLMAO against several baselines including Tarantula, Ochiai, DeepFL, and TRANSFER-FL.

Flaky Test Detection and Repair. Flaky test studies share two common datasets: IDoFT and FlakyCat.

IDoFT (International Dataset of Flaky Tests) contains 3,195 flaky and 618 non-flaky tests from Java and Python projects, categorised into order-dependent (OD), non-order-dependent (NOD), implementation-dependent (ID), and other types. It is used in S4 and F01.

FlakyCat contains 369 flaky tests from open-source Java projects, labelled by root cause category: Async Wait, Concurrency, Time, Test Order Dependency, and Unordered Collections. It is used in S4 and B03.

The main metric across these studies is F1 score, sometimes reported per category. Akli et al. [13] (B03) report a weighted F1 of 70% on FlakyCat. Fatima et al. [19] (F01) use CodeBLEU to measure repair quality, reporting a mean score of 81% when fix category labels are included in the prompt, compared to 75% without. Fix category prediction is also evaluated using precision (83%–98%) and recall (67%–97%).

Hashemi et al. [3] (S3) do not use a pre-labelled dataset. Instead, JS-TOD is evaluated on 81 open-source GitHub projects, measuring how many order-dependent flaky tests are detected across different reordering levels.

Vulnerability Detection. Khare et al. [6] (S6) use five security datasets: OWASP Benchmark (Java), Juliet Test Suite for C/C++ and Java, and CVEFixes, providing 5,000 code samples covering 25 CWE classes. The metrics are accuracy and F1 score, reported per vulnerability class. Results are also compared against CodeQL and two deep learning baselines, DeepDFA and LineVul. LLMs achieve a mean accuracy of 62.8% and F1 of 0.71 across all datasets. On real-world datasets, accuracy is 10.5% lower than on synthetic ones. LLMs outperform CodeQL on some vulnerability classes such as Path Traversal and OS Command Injection, while CodeQL performs better on Integer Overflow.

UI Test Repair. Xu et al. [7] (S7) evaluate on a standard Web UI test repair dataset containing 139 broken Selenium statements across multiple web applications. Metrics are reported at two levels: element matching accuracy and statement repair success rate. Baselines include WATER, VISTA, and EDIT DIS, as well as two more recent approaches, WEBEVO and SFTM.

Ding et al. [22] (E01) also show that evaluation design is a major issue in vulnerability detection. PrimeVul uses stricter labelling, de-duplication, and chronological splitting to make evaluation closer to real use cases. This matters because results on older datasets may be inflated by noisy labels or repeated examples.

3.5 RQ5: System Context and Practical Implications

In which system contexts are testing challenges studied, and what practical implications are reported?

The primary studies cover five system contexts: CI/CD pipelines, IoT platforms, microservice architectures, general open-source projects, and LLM-based testing environments. This section goes through each context and the practical implications reported.

CI/CD Pipelines. Gan et al. [1] (S1) examine GitHub repositories that use Selenium, Playwright, and Cypress within GitHub Actions workflows. Their study shows that although CI/CD adoption is increasing in open-source projects, the integration of UI testing remains relatively limited. Developers frequently reported that UI tests are expensive to maintain in CI/CD settings. They also highlighted unstable workflows and difficulty reproducing failures across different environments as major obstacles. These findings suggest that the availability of mature UI testing tools alone is not enough. In practice, maintenance effort and pipeline stability are also important factors when deciding how extensively UI testing should be integrated into CI/CD workflows.

IoT Platforms. Rodriguez-Cardenas et al. [2] (S2) examine OpenHAB and HomeAssistant, two large open-source IoT platforms. One of their clearest findings is the low level of test coverage. In HomeAssistant, 65% of add-ons have a test ratio below 0.5, while OpenHAB has an overall test ratio of only 0.04. Their developer survey also highlights three testing challenges that are particularly common in IoT systems and not well supported by existing tools. These include validating compatibility across different devices and protocols, debugging firmware updates that require access to physical hardware, and testing behaviour in low-power modes where assumptions about stable networks or predictable timing no longer hold. Overall, the study suggests that many current testing tools and practices do not transfer well to IoT development.

Microservice Architectures. Gregor et al. [8] (S9) develop a taxonomy of integration-relevant faults in microservice systems based on a literature review and expert interviews. A central finding is that testing problems in microservice environments cannot be addressed through better testing tools alone. The study identifies faults caused by technical issues, such as API contract mismatches and message format inconsistencies, but also faults linked to organisational factors, including weak team coordination and unclear service ownership. The proposed taxonomy offers practitioners a structured way to reason about common integration risks and to choose suitable testing techniques. However, the authors also note that the taxonomy has not yet been fully validated through large-scale industrial case studies.

General Open-Source Projects. Several studies examine general open-source Java, Python, and JavaScript projects, and their findings offer practical lessons for developers.

Luo et al. [11] (B01) analyse 201 flaky test fixing commits across 51 Apache open-source projects. Their results suggest several practical improvements for test design. For *Async Wait* flaky tests, the authors recommend explicit synchronisation mechanisms such as `waitFor` instead of fixed time delays. For *Test Order Dependency* cases, they recommend cleaning shared state in `setUp` and `tearDown` methods. The study also cites Google’s internal testing system, which experienced an average of 1.6 million test failures per day, with 4.56% attributed to flaky tests, illustrating the scale of the issue in large industrial environments.

Hashemi et al. [3] (S3) show that JavaScript developers using Jest face order-dependent flaky tests caused by shared mocking state. This cause had not been reported in earlier Java- or Python-based studies. Although Jest introduced a test randomisation feature in late 2023,

the authors note that it is not yet widely adopted. As a result, many projects may still contain hidden order-dependent tests that developers have not yet identified.

LLM-Based Testing Environments. Feldt et al. [21] (F03) discuss the practical challenges of using LLMs as autonomous testing agents. They identify three main barriers. First, context length limits make it difficult to analyse large codebases in a single interaction. Second, models typically do not retain memory across sessions, which makes longer testing workflows harder to manage. Third, LLMs need supporting middleware to connect with existing testing tools, build systems, and execution environments. The authors argue that progress in model capability alone is not sufficient, and that these surrounding engineering issues must also be solved before autonomous testing agents become practical.

Ji et al. [5] (S8) report a related limitation in the context of LLM-based fault localization. Because current models accept only limited input lengths, their approach operates at the method level rather than the full-project level. This restricts practical deployment, since real software systems are often much larger and require reasoning across multiple files, modules, and dependencies. ExecutionAgent [24] (E03) also highlights a practical issue in repository-level testing. In real projects, the difficult part is sometimes not analysing a failed test, but making the project build and the tests run. This requires handling dependencies, build tools, test frameworks, and environment-specific errors.

4 Related Work

Parry et al. [12] provided a comprehensive review of flaky testing research, summarizing the root causes, detection methods, remediation strategies, and impact on software testing practices. This study analyzed 76 peer-reviewed papers and pointed out that flaky testing has become a common problem across different programming languages and testing frameworks. However, this review primarily focused on flaky testing itself and did not cover rapidly developing areas such as fault localization, vulnerability detection, UI test repair, or autonomous testing agents based on Large Language Models (LLMs). Compared to existing reviews, this paper focuses more on the intersection between software testing challenges and recent LLM-based testing techniques. Specifically, this paper incorporates flaky tests, fault localization, vulnerability detection, UI test repair, and agentic testing workflows into a unified analytical framework. Furthermore, this paper discusses the practical directions represented by emerging agentic testing systems such as Glasswing and Mythos Preview, topics that have received relatively little systematic discussion in existing survey literature.

5 Discussion

5.1 Overview of the Current Research Landscape

The 25 papers included in the analysis reveal that current software testing research primarily focuses on several areas: flaky test detection and remediation, UI test maintenance, fault localization, security vulnerability detection, and testing issues in complex systems such as CI/CD, IoT, and microservices [11, 12, 3, 19, 1, 7, 18, 15, 16, 5, 6, 2, 8]. This research indicates that modern software testing is no longer simply about determining whether a test passes or fails; it is related to the execution environment, shared state, system architecture, development processes, and toolchains.

Methodologically, existing research exhibits a clear trend: a shift from traditional test execution-dependent methods to LLM-assisted methods. Traditional methods, such as SBFL, rely on test coverage and pass/fail results to locate suspicious code [15, 16, 4]; empirical research typically summarizes problems through GitHub repository mining, commit analysis, developer

surveys, and manual classification [11, 12, 18, 1]. In recent years, LLMs have been used for fault localization, vulnerability detection, flaky test classification, and test repair, demonstrating their potential to reduce manual analysis and dependence on test execution [5, 20, 6, 13, 14, 19].

However, most of these studies currently address single tasks. For example, some only perform detection, some only perform classification, some only perform localization, and some only perform repair. They have not yet formed a complete workflow that connects failure observation, context collection, root-cause analysis, patch generation, and validation. Therefore, the main problem with existing research is not a lack of individual technologies, but rather a lack of an end-to-end intelligent testing workflow that can run in a real development environment.

5.2 Research Gaps Across Methodological Themes

The first research gap lies in fault localization. Traditional SBFL methods, such as Tarantula (B05) and Ochiai (B06), can calculate suspiciousness scores for code locations based on test coverage and pass/fail results, offering good interpretability [15, 16]. However, these methods heavily rely on reliable test cases and coverage data. While S5 (Applis et al.) improved SBFL performance through compiler information, type information, and AST-based filtering, it still fundamentally remains dependent on test execution [4]. LLM-based fault localization attempts to reduce this dependency: S8 (Ji et al.) predicts fault lines through fine-tuning code LLMs, and F02 (LLMAO) further explores test-free fault localization [5, 20]. However, these methods are primarily validated on benchmarks like Defects4J, lacking systematic evaluation in real-world production environments. Therefore, the key gap is whether LLM-based fault localization can work stably in large codebases, noisy logs, incomplete testing, and continuously evolving real-world projects—evidence remains scarce.

The second research gap lies in vulnerability detection. S6 (Khare et al.) compared the impact of Basic prompts, CWE-specific prompts, and Dataflow prompts on 16 LLMs, demonstrating that prompt design significantly affects vulnerability detection performance [6]. The Dataflow prompt performed better on some real-world datasets, but the overall results remained limited: the average accuracy was 62.8%, the average F1 score was 0.71, and performance on real-world datasets was lower than on synthetic datasets [6]. This indicates that the vulnerability detection capability of current LLMs in real-world code remains unstable. Furthermore, the experiments primarily focused on known CWE types and labeled datasets; therefore, the ability of LLMs to discover unknown or project-specific vulnerabilities remains unclear.

The third research gap lies in flaky testing and empirical testing research. LLM-based flaky testing methods have evolved from prediction to classification and remediation: B04 (Flakify) determines whether a test is flaky, B03 and S4 (FlakyXbert) predict the flaky test category, and F01 (FlakyFix) further utilizes the fix category to guide LLMs in generating repair code [14, 13, 19]. However, these methods typically rely on labeled datasets such as IDoFT and FlakyCat, and their cross-project generalization ability remains uncertain. Meanwhile, empirical research on flaky tests, UI testing, IoT platforms, and microservices provides valuable taxonomies and practical observations, but most remain descriptive [11, 12, 3, 1, 7, 18, 2, 8]. They explain the problems but do not provide an end-to-end pipeline that automatically connects detection, localization, explanation, repair, and validation.

5.3 Glasswing as an Emerging Practical Reference

Glasswing provides an important practical reference for LLM-based testing and agent-based software analysis. In April 2026, Anthropic released Project Glasswing, a critical software security initiative centered around Claude Mythos Preview, aiming to help protect real-world critical software systems. Unlike most LLM-based approaches in SLR that rely on fixed benchmarks or single-task settings, Glasswing’s public technical reports shift the focus from benchmark-based evaluation to real-world zero-day vulnerability discovery. The report describes an agentic

scaffold: the system runs the target program and its source code in a container isolated from the internet, then calls Mythos Preview via Claude Code, providing a simple instruction like "Please find a security vulnerability in this program." Subsequently, the model autonomously reads the code, proposes vulnerability hypotheses, runs the target program or uses debugging tools for verification, and outputs a structured report containing proof-of-concept exploits and reproduction steps when a problem is found. After the initial scan, the system will launch another Mythos Preview agent to further verify the report, determining whether the vulnerability is real and has actual impact. This "agentic discovery + independent verification" structure has not yet become the main research paradigm in the studies included in this SLR.[40, 41]

From the research gaps summarized in Section 4.2, Glasswing’s significance lies not in its complete solution to these problems, but in demonstrating a direction that is closer to real-world environments. First, compared to the reliance of traditional SBFL on test coverage and pass/fail results, Glasswing points to an analytical approach that is not limited to coverage-based suspiciousness ranking: the model can combine source code reading, runtime experiments, debugging tools, and security reasoning to discover problems. Second, compared to LLM-based fault localization like S8 and F02, which are mainly validated on academic benchmarks such as Defects4J, Glasswing’s public case studies are closer to real-world large-scale software systems and critical infrastructure environments. For example, Mythos Preview was reported to have discovered a 27-year-old OpenBSD SACK vulnerability, a 16-year-old FFmpeg H.264 decoder vulnerability, and a 17-year-old FreeBSD NFS remote code execution vulnerability. Third, compared to S6’s vulnerability detection focused on known CWE types and labeled datasets, Glasswing places greater emphasis on previously unknown zero-day vulnerabilities rather than simply identifying samples from known categories. Furthermore, the report shows that in 198 manually reviewed vulnerability reports, expert severity assessments were 89% identical to Claude’s, and 98% were within the same severity level. This indicates that agentic LLMs can not only generate vulnerability hypotheses but may also support vulnerability impact assessments to some extent.[6, 41]

5.4 Consolidated Research Gaps

Based on the methodological limitations identified in existing literature in Section 4.2 and the open issues exposed by Glasswing in Section 4.3, three core research gaps can be further summarized.

The first gap is the lack of validation in real-world repository environments for LLM-based fault localization. Existing research (S8, F02) has demonstrated that LLMs can predict fault locations with reduced or even no reliance on test execution, but these experiments are primarily conducted on academic benchmarks such as Defects4J [5, 20]. Meanwhile, Glasswing has shown the potential of agentic LLMs to discover security vulnerabilities in large real-world codebases and critical software infrastructure, but its validation scope is mainly limited to vulnerability discovery, not general production bug localization [40, 41]. Therefore, whether LLM-based fault localization can reliably locate common application-level bugs from real repository-level information—such as issue descriptions, source-code context, commit history, and verified bug-fixing commits—still lacks empirical evidence.

The second gap is that the agentic scaffold demonstrated by Glasswing has not been fully applied to ordinary fault localization scenarios. Glasswing’s agentic architecture enables the model to autonomously read code, form hypotheses, run verifications, and generate vulnerability reports, achieving significant results in security vulnerability discovery [41]. However, many faults in real-world business systems are not memory corruption, buffer overflows, or sandbox escapes, but rather business logic errors, type errors, state synchronization anomalies, data processing anomalies, or API usage errors. These types of bugs differ significantly from security vulnerabilities in their triggering methods, location clues, and remediation methods; therefore, it cannot be directly assumed that Glasswing’s agentic approach is equally applicable to ordinary

application-level bug detection and fault localization.

The third gap is that the impact of context level and independent verification on fault localization has not been systematically evaluated. Glasswing uses a second independent agent to verify initial vulnerability reports; meanwhile, Carlini et al. reported that in 198 manually reviewed vulnerability reports, the model’s severity assessment was exactly consistent with expert assessment in 89% of cases, and within one severity level in 98% of cases [41]. It is important to note that this figure reflects the consistency of severity assessment, not the accuracy of fault localization. On the other hand, S6’s experiments show that prompt design significantly impacts LLM-based vulnerability detection performance, with an overall average accuracy of 62.8% [6]. These results collectively suggest that the contextual form of model input and the verification mechanism may significantly affect the quality of LLM judgments. However, there is currently no research comparing the impact of different context levels—such as error messages, stack traces, related code snippets, issue context, and historical fix commits—or single-agent versus independent verification modes on fault localization accuracy in ordinary fault localization scenarios.

Table 8: Consolidated research gaps and their sources.

Gap	Description	Source
Gap A	LLM-based fault localization lacks real repository-level validation.	SLR Theme 2 (S8, F02) and comparison with Glasswing.
Gap B	Agentic scaffolds have not been sufficiently applied to ordinary application-level bugs.	Limitation of Glasswing’s current focus on security vulnerabilities.
Gap C	The effects of context level and independent verification on fault localization remain unclear.	S6 prompt results and Glasswing’s independent verification mechanism.

Together, these gaps point to a central unresolved problem: existing research and practice have separately demonstrated LLM-based fault localization on academic benchmarks, agentic vulnerability discovery in real-world codebases, and the importance of prompt and context design for model performance. However, the intersection of these directions remains underexplored.

6 Responsible Use of AI Tools

This study utilized AI-assisted tools in certain stages. During the literature review phase, AI was used to assist in summarizing basic information from candidate papers, such as research topics, methods, and key findings, helping authors to conduct preliminary work more efficiently. The final inclusion and exclusion decisions were not made by AI but independently by the authors based on predefined inclusion and exclusion criteria.

During the results writing phase, AI was used to assist in organizing the relevant content from included papers into more clearly structured paragraphs. Authors used original paper content, reading notes, and intermediate summaries as input; AI primarily generated preliminary expressions and paragraph structures. Subsequently, authors verified the accuracy of all factual statements, data, citations, and interpretations against the original papers.

During the writing and formatting phase, AI was also used to assist in polishing English academic expression, adjusting LaTeX table formatting, and generating formatted BibTeX entries. All AI-generated or modified content was manually reviewed and revised by the authors.

Overall, the AI tools were used only as writing support, structuring, and formatting assistance. The definition of the research question, the design of the analytical framework, the

decision-making process for literature selection, the interpretation of research results, and the formation of the final research gaps and conclusions were all completed independently by the authors. AI was not used to replace research judgment, nor was it used to generate unverified academic citations, experimental evidence, or research conclusions.

7 Conclusion

This study reviews 25 papers on software testing challenges and LLM software testing techniques. Overall, existing research covers flaky tests, UI testing vulnerabilities, fault location, vulnerability detection, test remediation, and testing issues in CI/CD, IoT, microservices, and industrial systems.

The results show that modern software testing problems are often highly environment-dependent. Flaky tests can be caused by asynchronous behavior, concurrency, test order dependencies, or shared state issues in specific frameworks, such as shared mocking state in JavaScript testing. UI testing vulnerabilities mainly stem from frequent changes to web interfaces, while IoT and microservice systems are also affected by hardware differences, distributed execution, and team collaboration issues.

Methodologically, research trends have also changed significantly. Early methods relied primarily on test execution, coverage data, repeated runs, and manual analysis. In recent years, LLM has been increasingly used for fault location, vulnerability detection, flaky test classification, UI test remediation, and test execution in real code repositories. However, these methods still have limitations. Many studies still primarily evaluate on academic benchmarks, rely on labeled datasets, or address single tasks rather than forming complete testing workflows.

Therefore, the most significant research gap lies in the lack of sufficient evidence to demonstrate the stable performance of LLM-based and agentic testing methods in real-world development environments. While existing research has shown potential on benchmarks and specific tasks, understanding of their performance with large-scale codebases, incomplete logs, constantly changing dependencies, and real CI/CD constraints remains limited. This provides direction for future research: evaluating the practical effectiveness of LLM-based fault localization in real repository environments, leveraging real bug reports, source-code context, repository history, and validated bug-fixing commits.

References

- [1] Xiaoxiao Gan, Huayu Liang, and Chris Brown. “Challenges, Strategies, and Impacts: A Qualitative Study on UI Testing in CI/CD Processes from GitHub Developers’ Perspectives”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 186–197.
- [2] Daniel Rodríguez-Cardenas et al. “Testing Practices, Challenges, and Developer Perspectives in Open-Source IoT Platforms”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 290–301.
- [3] Negar Hashemi et al. “Detecting and Evaluating Order-Dependent Flaky Tests in JavaScript”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 13–24.
- [4] Leonhard Applis, Matthías Páll Gissurarson, and Annibale Panichella. “Suspicious Types and Bad Neighborhoods: Filtering Spectra with Compiler Information”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 233–243.

- [5] Suhwan Ji et al. “Impact of Large Language Models of Code on Fault Localization”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 302–313.
- [6] Avishree Khare et al. “Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.
- [7] Zhuolin Xu, Qiushi Li, and Shin Hwei Tan. “Understanding and Enhancing Attribute Prioritization in Fixing Web UI Tests with LLMs”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 326–337.
- [8] Lena Gregor et al. “A Taxonomy of Integration-Relevant Faults for Microservice Testing”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 138–149.
- [9] Barbara Kitchenham et al. “Systematic Literature Reviews in Software Engineering – A Systematic Literature Review”. In: *Information and Software Technology* 51.1 (2009), pp. 7–15.
- [10] Riddhi More and Jeremy S. Bradbury. “An Analysis of LLM Fine-Tuning and Few-Shot Learning for Flaky Test Detection and Classification”. In: *Proceedings of the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 349–359.
- [11] Qingzhou Luo et al. “An Empirical Analysis of Flaky Tests”. In: *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. ACM, 2014, pp. 643–653.
- [12] Owain Parry et al. “A Survey of Flaky Tests”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31.2 (2022), 17:1–17:74.
- [13] Amal Akli et al. “FlakyCat: Predicting Flaky Tests Categories using Few-Shot Learning”. In: *Proceedings of the 4th ACM/IEEE International Workshop on Automated Software Testing (AST)*. IEEE/ACM, 2023, pp. 140–151.
- [14] Sakina Fatima, Taher Ahmed Ghaleb, and Lionel Briand. “Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests”. In: *IEEE Transactions on Software Engineering* 49.4 (2023), pp. 1912–1927.
- [15] James A. Jones, Mary Jean Harrold, and John T. Stasko. “Visualization of Test Information to Assist Fault Localization”. In: *Proceedings of the 24th International Conference on Software Engineering (ICSE)*. ACM, 2002, pp. 467–477.
- [16] Rui Abreu et al. “A Practical Evaluation of Spectrum-Based Fault Localization”. In: *Journal of Systems and Software* 82.11 (2009), pp. 1780–1792.
- [17] Zhangyin Feng et al. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. ACL, 2020, pp. 1536–1547.
- [18] Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. “Why Do Record/Replay Tests of Web Applications Break?” In: *Proceedings of the 9th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2016, pp. 180–190.
- [19] Sakina Fatima, Hadi Hemmati, and Lionel C. Briand. “FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair”. In: *IEEE Transactions on Software Engineering* 50.12 (2024), pp. 3146–3171.
- [20] Aidan Z. H. Yang et al. “Large Language Models for Test-Free Fault Localization”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM/IEEE, 2024, 17:1–17:12.

- [21] Robert Feldt et al. “Towards Autonomous Testing Agents via Conversational Large Language Models”. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE/ACM, 2023, pp. 1688–1693.
- [22] Yangruibo Ding et al. “Vulnerability Detection with Code Language Models: How Far Are We?”. In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM/IEEE, 2025, pp. 1729–1741. DOI: 10.1109/ICSE55347.2025.00038.
- [23] Bozhi Wu et al. “Enhancing Vulnerability Detection via Inter-Procedural Semantic Completion”. In: *Proceedings of the ACM on Software Engineering 2.ISSTA (2025)*. DOI: 10.1145/3728912.
- [24] Islem Bouzenia and Michael Pradel. “You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects”. In: *Proceedings of the ACM on Software Engineering 2.ISSTA (2025)*. DOI: 10.1145/3728922.
- [25] Alexander Berndt et al. “On the Flakiness of LLM-Generated Tests for Industrial and Open-Source Database Management Systems”. In: *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM/IEEE, 2026, pp. 1–12. DOI: 10.1145/3786583.3786919.
- [26] Alexander Berndt, Thomas Bach, and Sebastian Baltes. “Do Test and Environmental Complexity Increase Flakiness? An Empirical Study of SAP HANA”. In: *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM/IEEE, 2024, pp. 572–581. DOI: 10.1145/3674805.3695407.
- [27] Wing Lam et al. “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests”. In: *Proceedings of the 12th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2019, pp. 312–322.
- [28] Alex Gyori et al. “Reliable Testing: Detecting State-Polluting Tests to Prevent Test Dependency”. In: *Proceedings of the 24th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2015, pp. 223–233. DOI: 10.1145/2771783.2771793.
- [29] Wing Lam et al. “Root Causing Flaky Tests in a Large-Scale Industrial Setting”. In: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2019, pp. 101–111.
- [30] Abdulrahman Alshammari et al. “FlakeFlagger: Predicting Flakiness Without Rerunning Tests”. In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2021, pp. 1572–1584. DOI: 10.1109/ICSE43902.2021.00140.
- [31] Qusay Idrees Sarhan and Árpád Beszédes. “A Survey of Challenges in Spectrum-Based Software Fault Localization”. In: *IEEE Access* 10 (2022), pp. 10618–10639. DOI: 10.1109/ACCESS.2022.3144079.
- [32] Daya Guo et al. “UniXcoder: Unified Cross-Modal Pre-training for Code Representation”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL, 2022, pp. 7212–7225. DOI: 10.18653/v1/2022.acl-long.499.
- [33] Guru Bhandari, Amara Naseer, and Leon Moonen. “CVEFixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software”. In: *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. ACM, 2021, pp. 30–39. DOI: 10.1145/3475960.3475985.
- [34] Yaqin Zhou et al. “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks”. In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. Curran Associates, 2019, pp. 10197–10207.

- [35] Shaurya Roy Choudhary et al. “WATER: Web Application Test Repair”. In: *Proceedings of the First International Workshop on End-to-End Test Script Engineering (ETSE)*. ACM, 2011. DOI: 10.1145/2002931.2002935.
- [36] Amir Makhshari and Ali Mesbah. “IoT Bugs and Development Challenges”. In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2021, pp. 460–472. DOI: 10.1109/ICSE43902.2021.00051.
- [37] Ahmadreza Saboor Yaraghi et al. “Automated Test Case Repair Using Language Models”. In: *IEEE Transactions on Software Engineering* 51.4 (2025), pp. 1104–1133. DOI: 10.1109/TSE.2025.3541166.
- [38] Saikat Dutta, August Shi, and Sasa Misailovic. “FLEX: Fixing Flaky Tests in Machine Learning Projects by Updating Assertion Bounds”. In: *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2021, pp. 603–614.
- [39] Chengpeng Li et al. “Systematically Producing Test Orders to Detect Order-Dependent Flaky Tests”. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2023, pp. 627–638.
- [40] Anthropic. *Project Glasswing*. Accessed: 2026-04-28. 2026. URL: <https://www.anthropic.com/glasswing>.
- [41] Nicholas Carlini et al. *Mythos Preview: Autonomous Vulnerability Discovery*. Anthropic Frontier Red Team Technical Report. 2026. URL: <https://red.anthropic.com/2026/mythos-preview>.